Linked Lists

Comp Sci 214, Spring 2025

Don’t forget to check in on Youhere!

Today

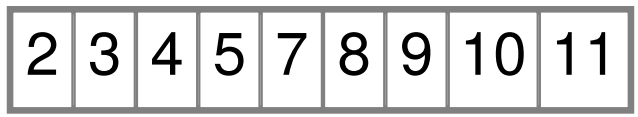
- Try out the YouHere app; we start counting next time
- Hand in your signed academic integrity policy

- DSSL2 Q&A
- Linked lists

DSSL2 Q&A

Linked lists

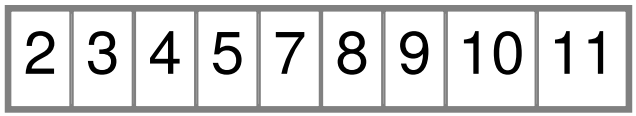
The problem with vectors



What if we want to add 6 between 5 and 7?

Can't do that! Vector is full. What do we do?

The problem with vectors



What if we want to add 6 between 5 and 7?

Can't do that! Vector is full. What do we do?

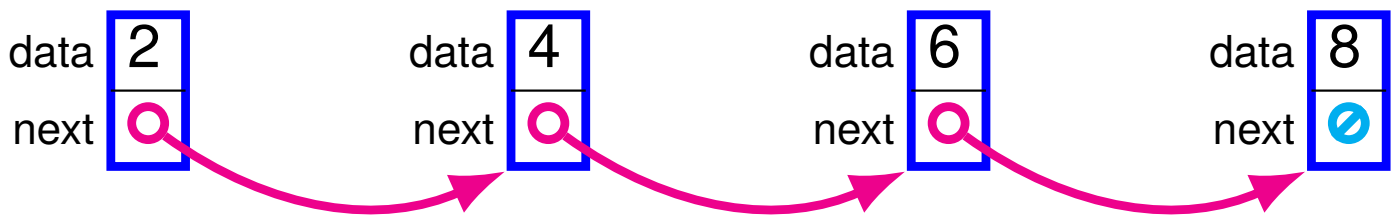
Need to create a new, bigger vector, and copy everything over…

Instead: A Treasure Hunt for Books!

- Leave books all around the house
- Each book has a sticky note with the location of the next
- Follow the clues until you find the book you want!

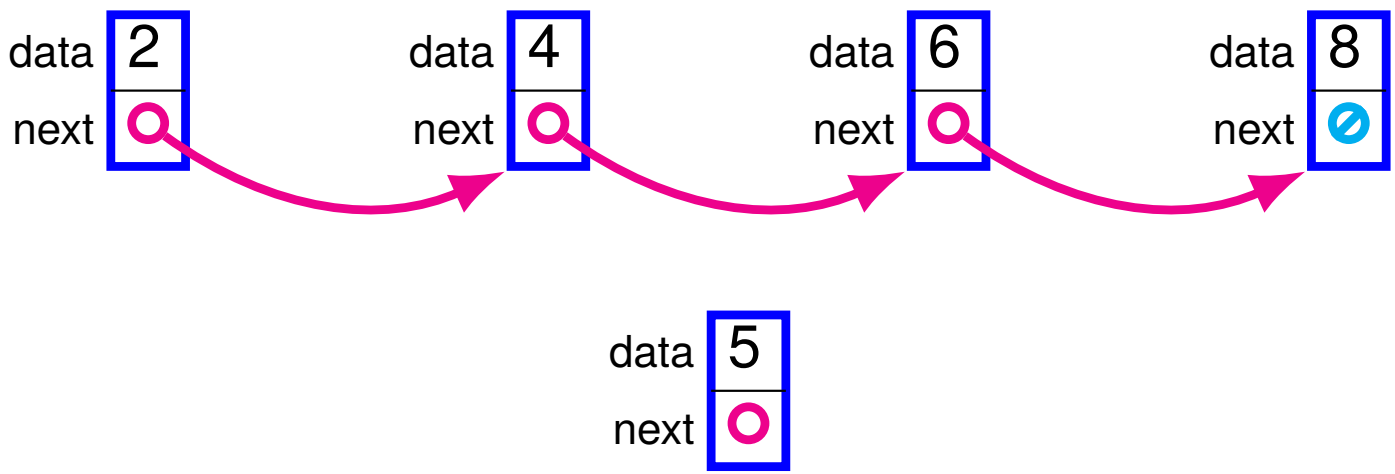
Linked lists

- You've seen those in 111, but they looked a bit different
  - `cons` creates a new linked list node, to hold a new element
    - (sometimes called `node` or `link`)
  - The `data` field holds one data element
    - (sometimes called `car` or `first`)
  - The `next` field holds an arrow to the next node in the list
    - (sometimes called `cdr` or `rest`)



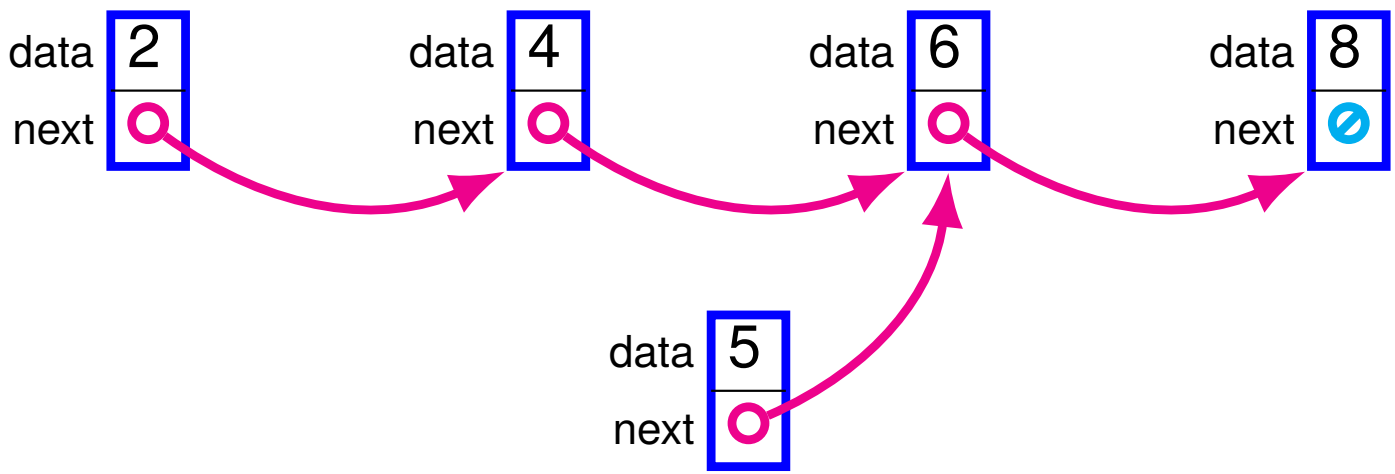
Linked lists

- Inserting in the middle? No problem!
  - Just change the arrows



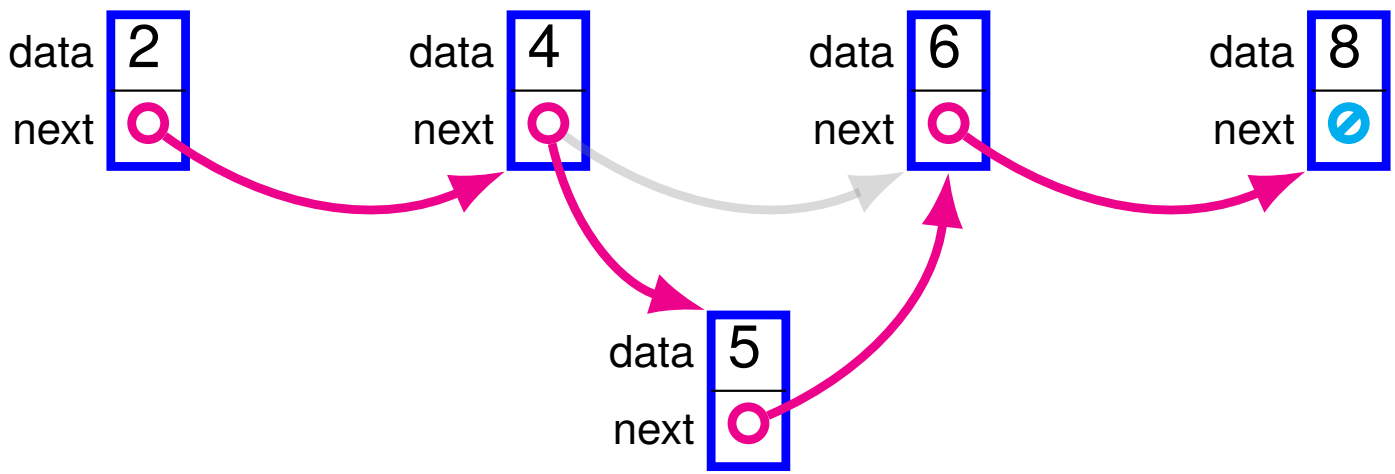
Linked lists

- Inserting in the middle? No problem!
  - Just change the arrows



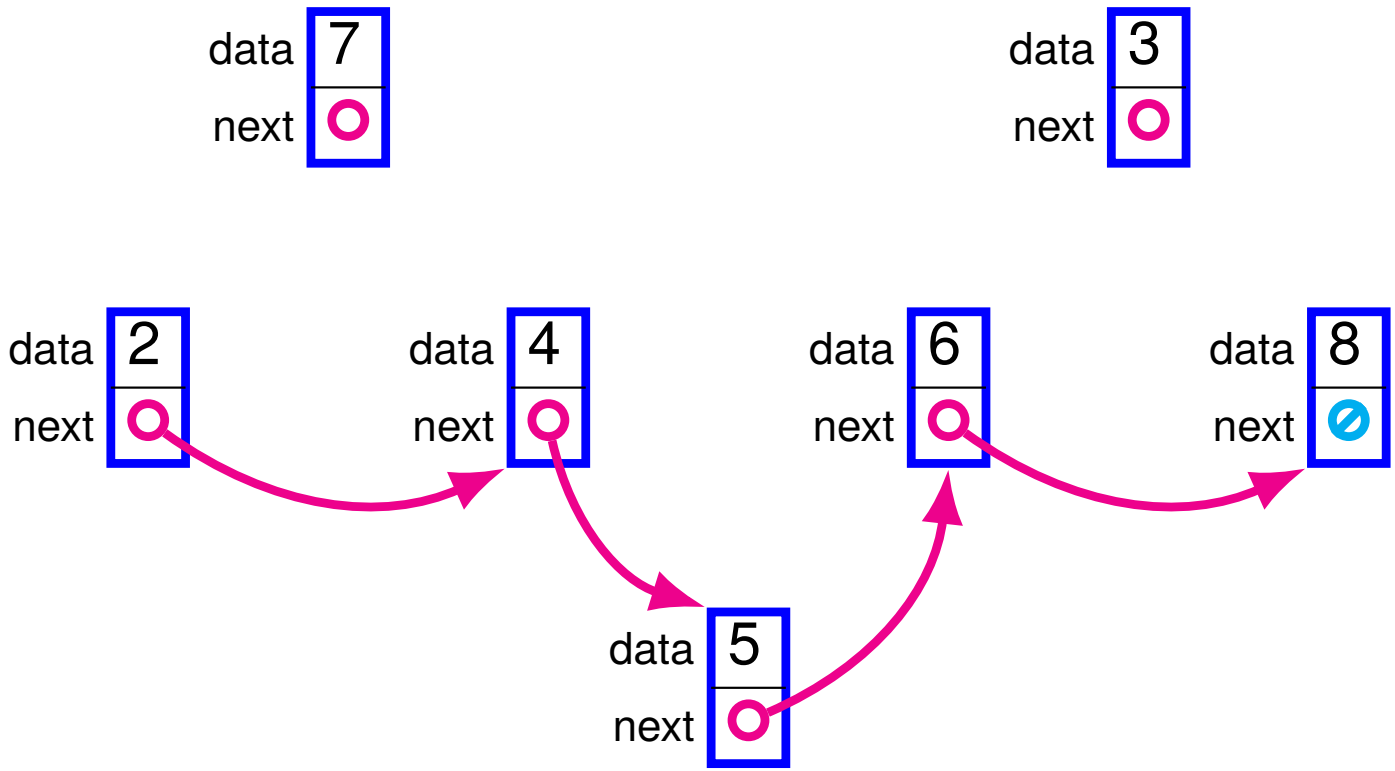
Linked lists

- Inserting in the middle? No problem!
  - Just change the arrows



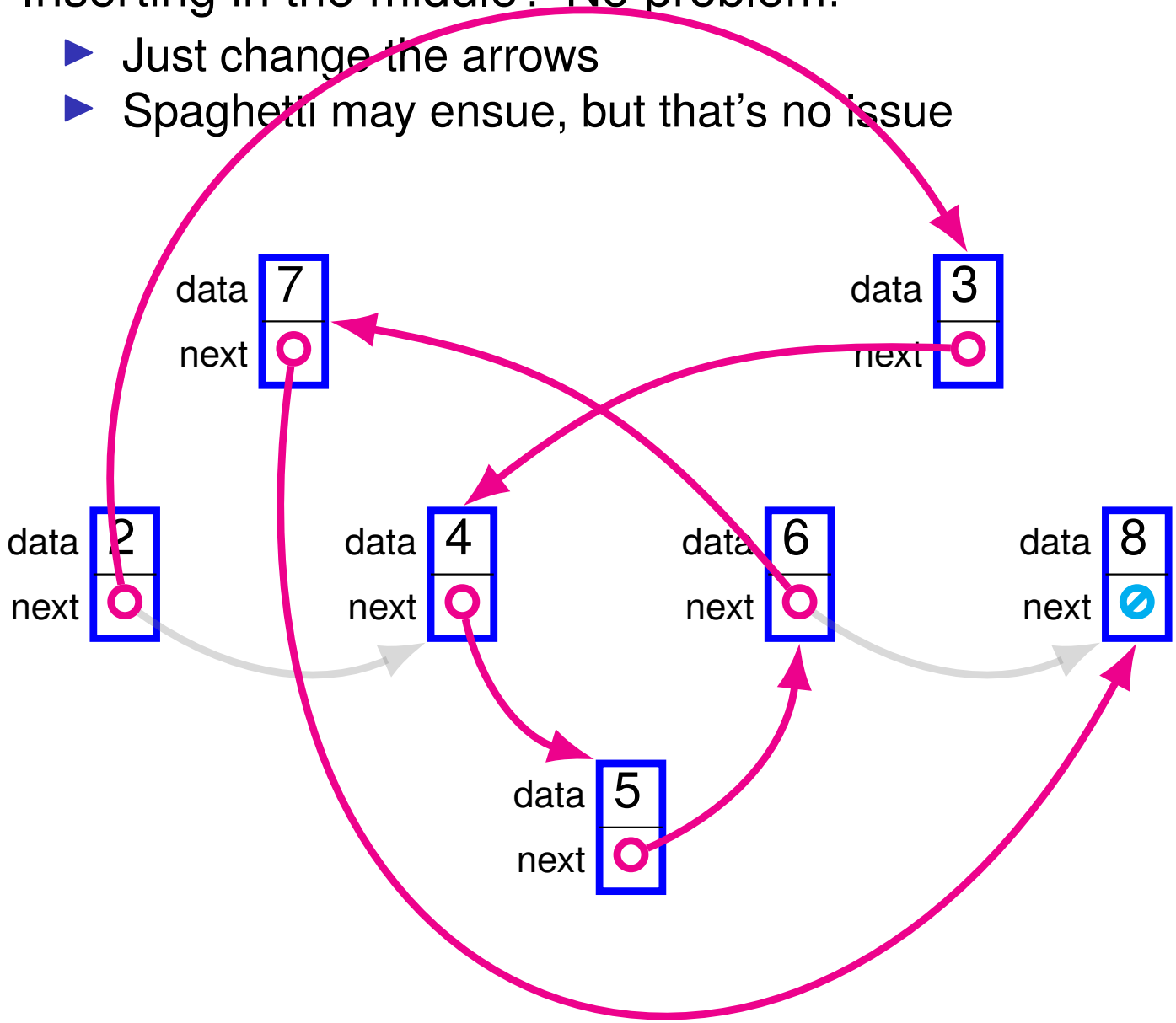
Linked lists

- Inserting in the middle? No problem!
  - Just change the arrows



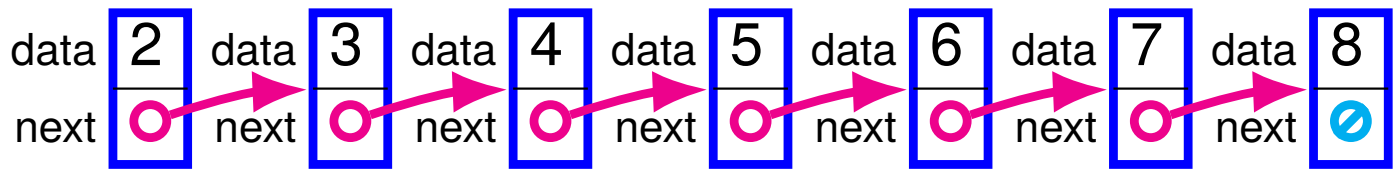
Linked lists

- Inserting in the middle? No problem!
  - Just change the arrows
  - Spaghetti may ensue, but that's no issue



Inserting at the beginning

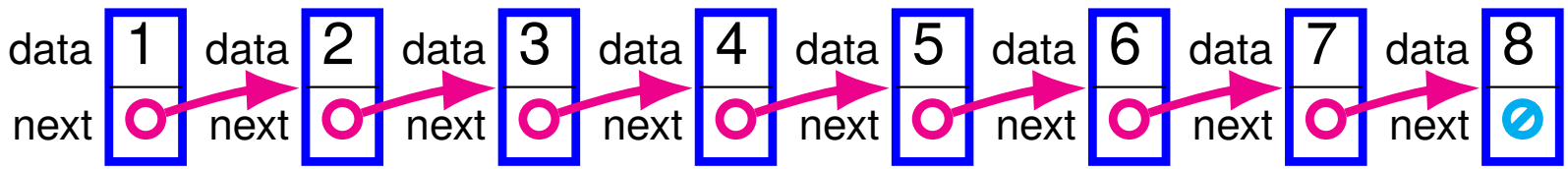
node2



- Even easier! Just add a new node

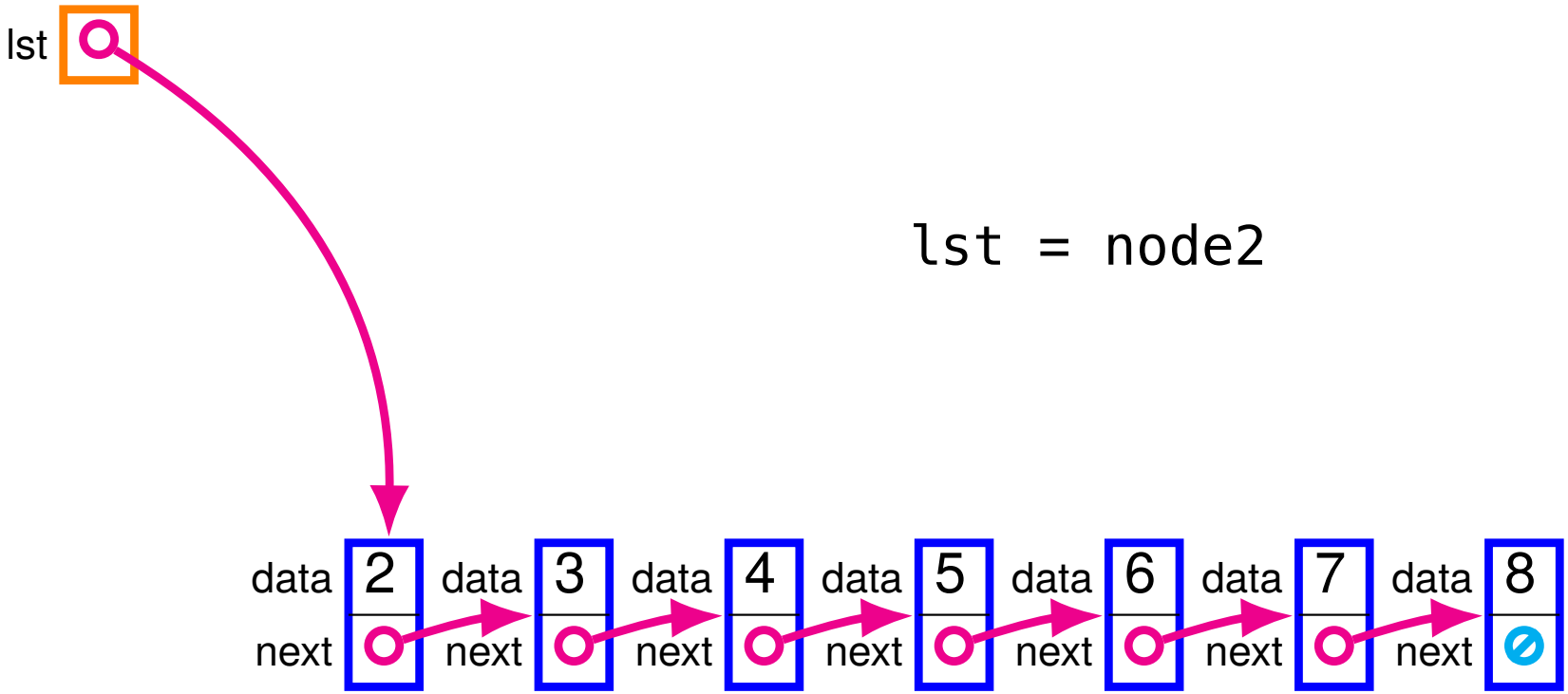
Inserting at the beginning

```
cons(1, node2)
```



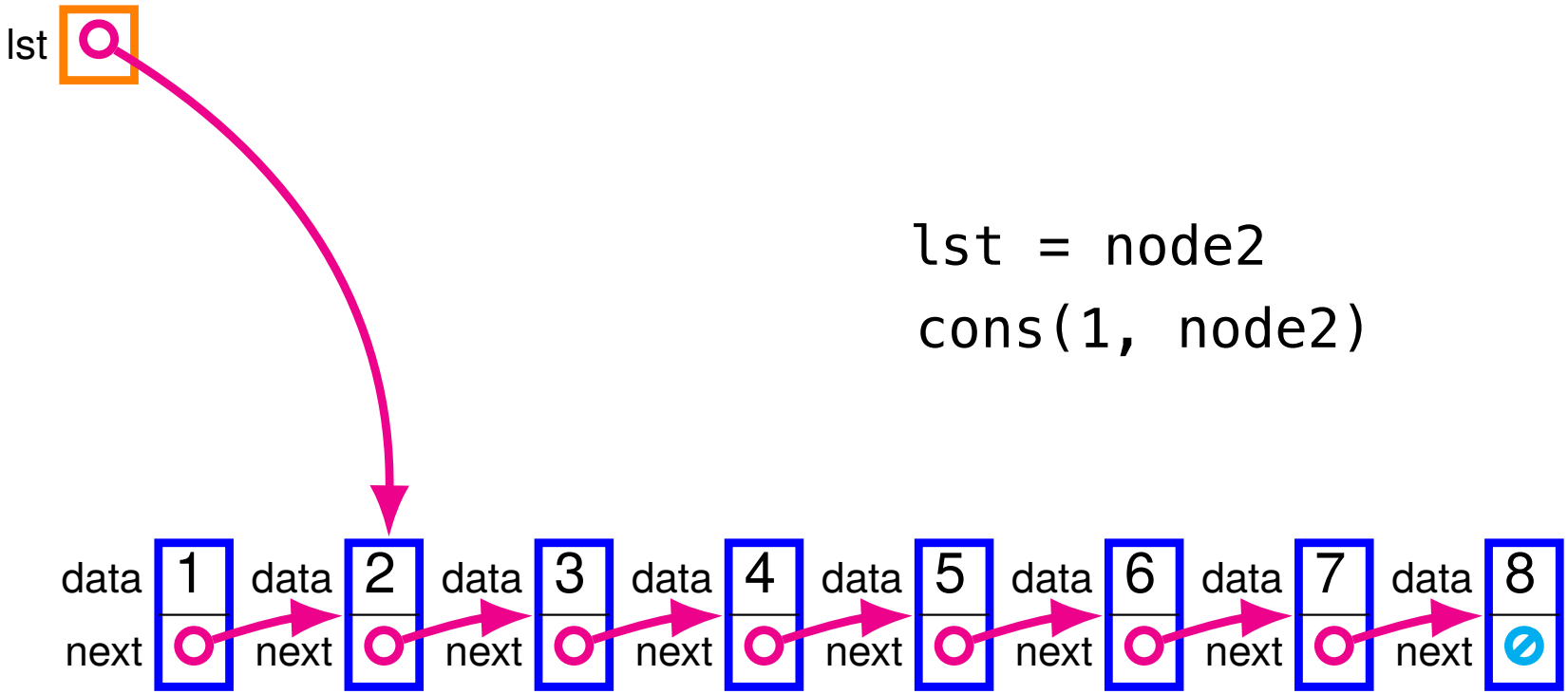
- Even easier! Just add a new node

Inserting at the beginning



- Now let's have a variable holding the start of our list

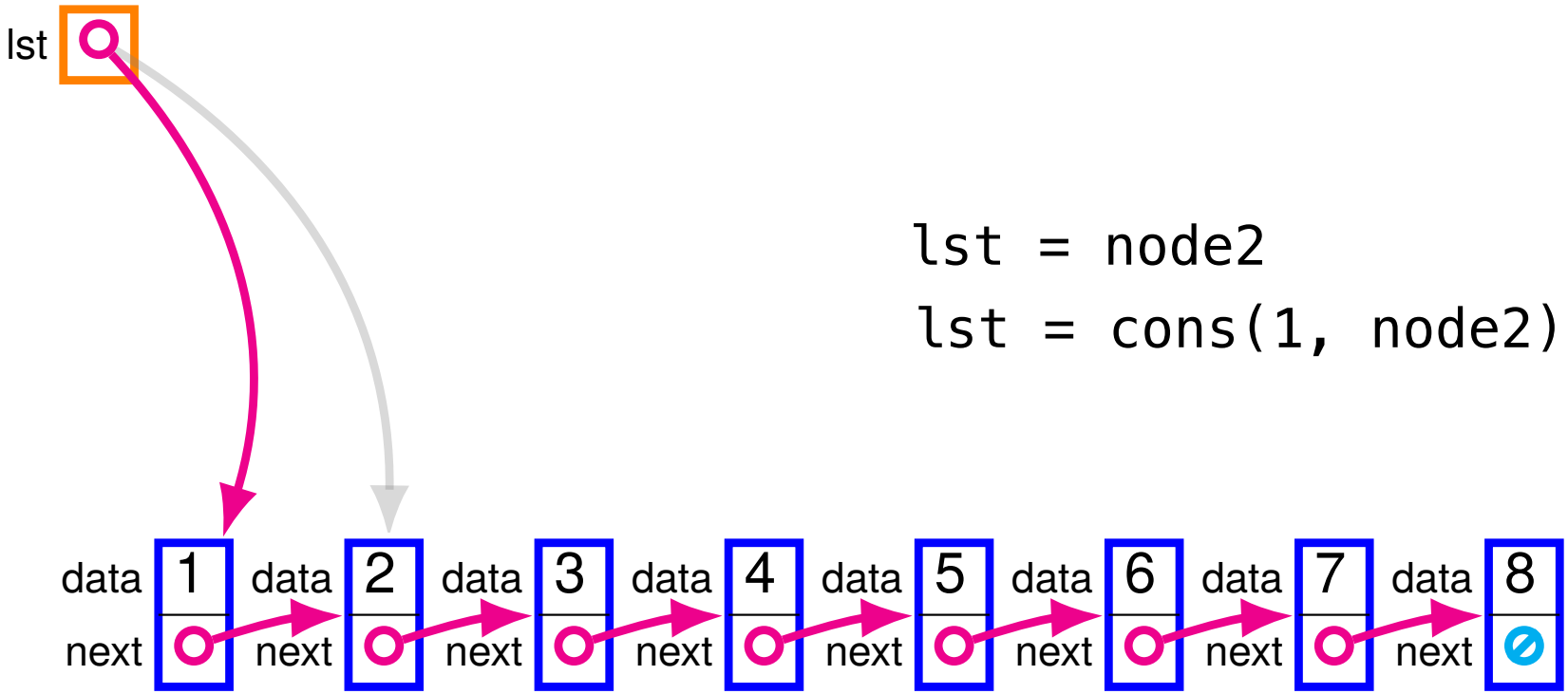
Inserting at the beginning




```
lst = node2
cons(1, node2)
```

- Now let's have a variable holding the start of our list
- Oh, but what used to be the start isn't the start anymore...

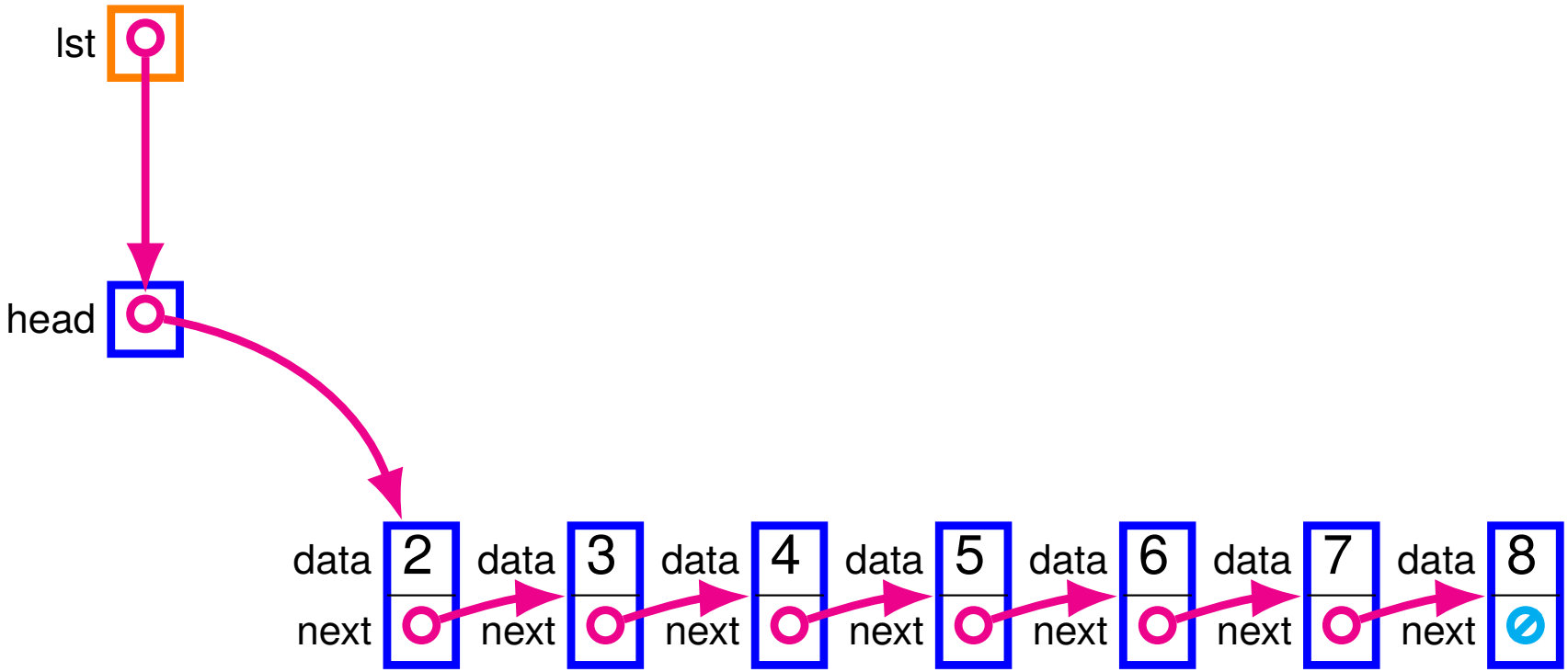
Inserting at the beginning




```
lst = node2
lst = cons(1, node2)
```

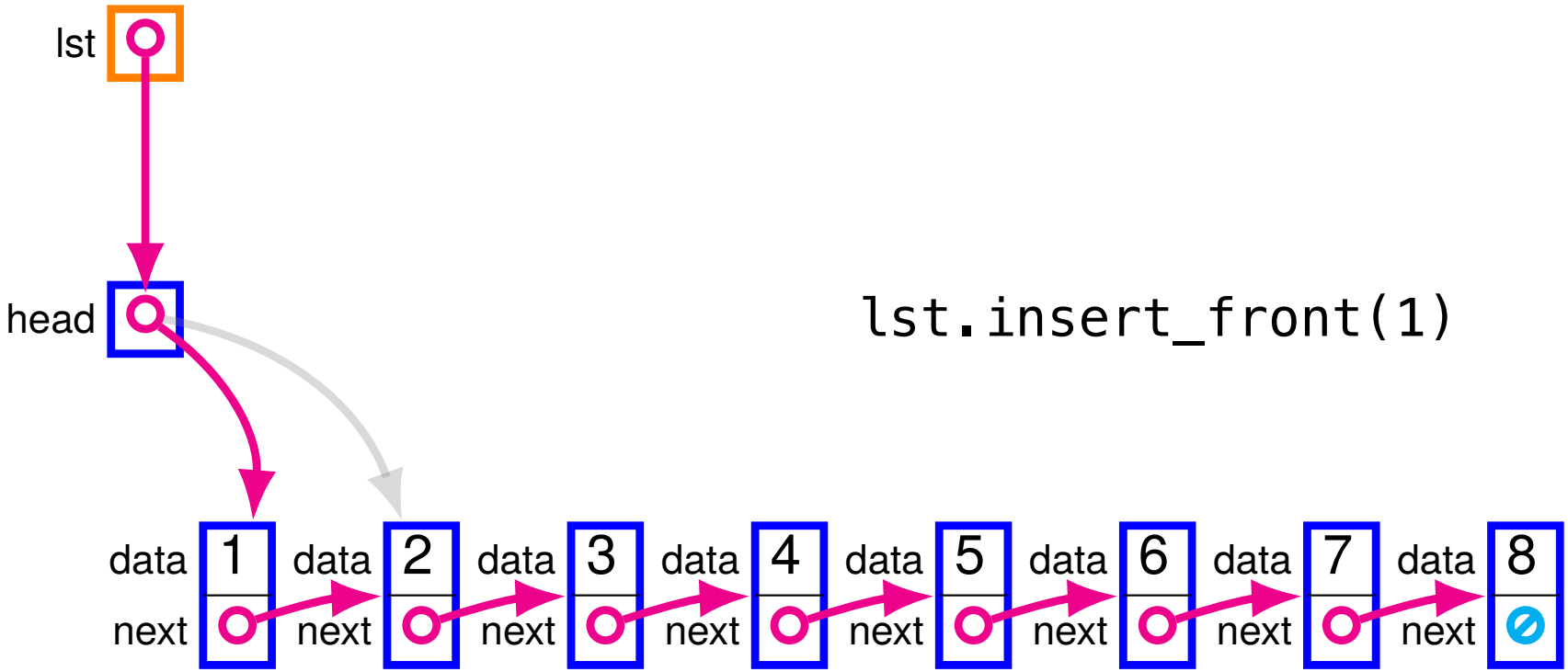
- Now let's have a variable holding the start of our list
- Oh, but what used to be the start isn't the start anymore...
  - Need to change the variable, too!
  - That's a bug waiting to happen

Indirection



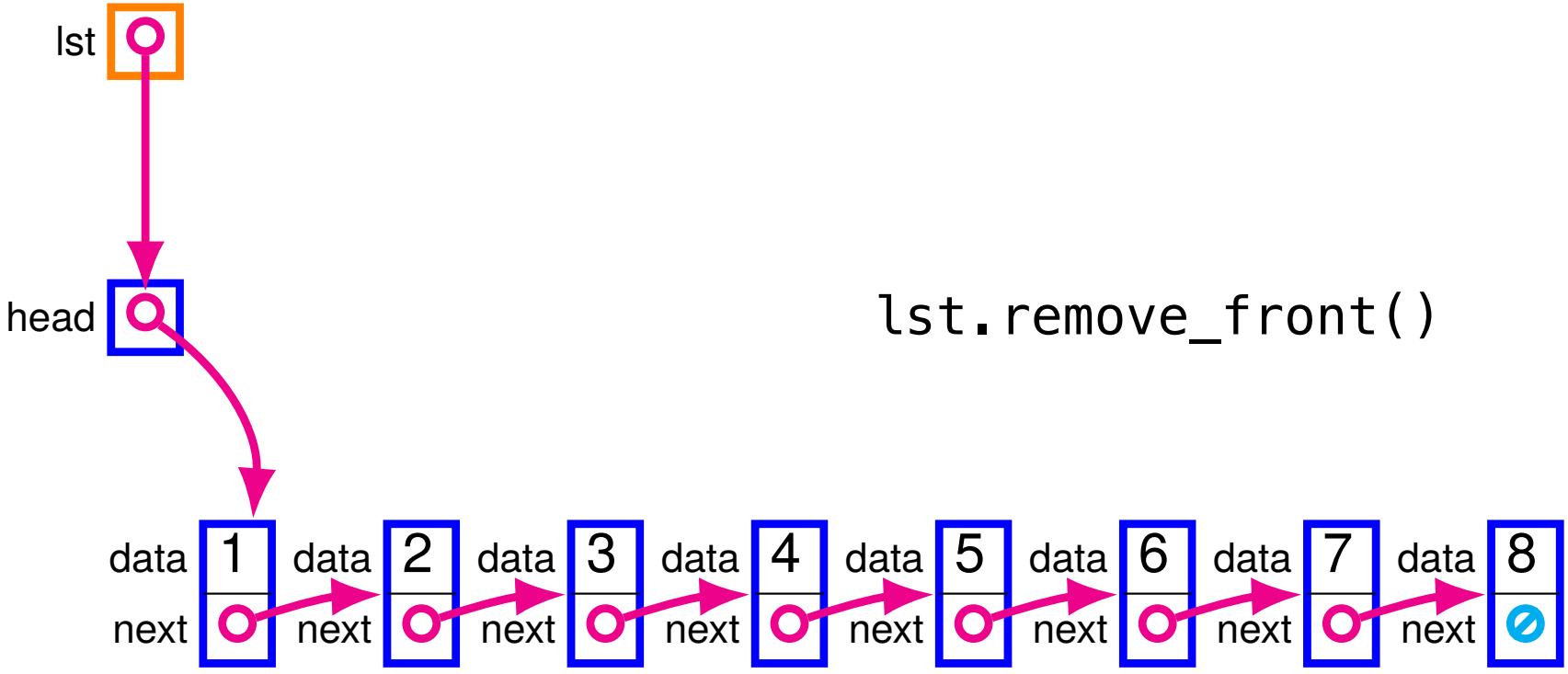
- Better: have the list be a *header* pointing to the first node
  - Then can change the header, not the variable!
  - Header is within the DS, DS operations can change it!
  - Variable is outside the DS, must rely on client to change…

Indirection



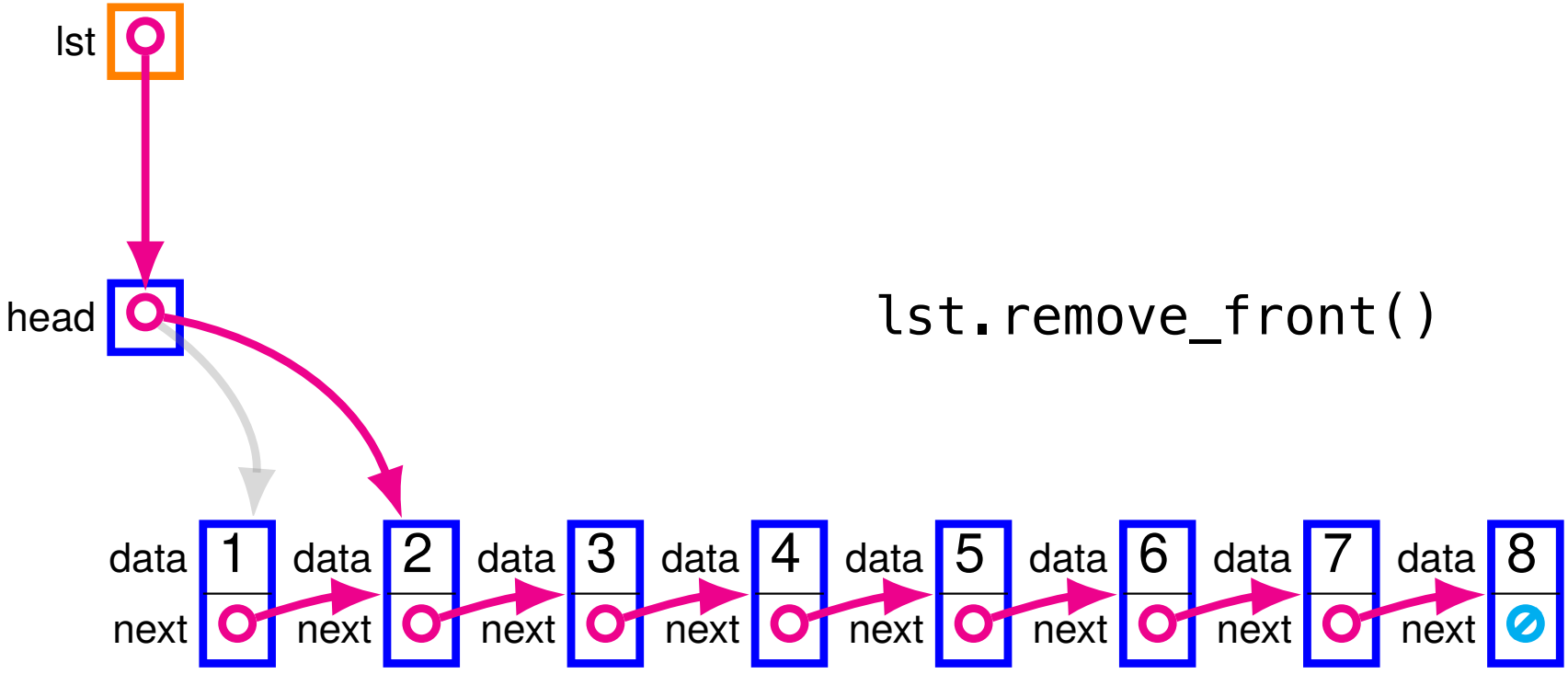
- Better: have the list be a *header* pointing to the first node
  - Then can change the header, not the variable!
  - Header is within the DS, DS operations can change it!
  - Variable is outside the DS, must rely on client to change…

Removing at the beginning



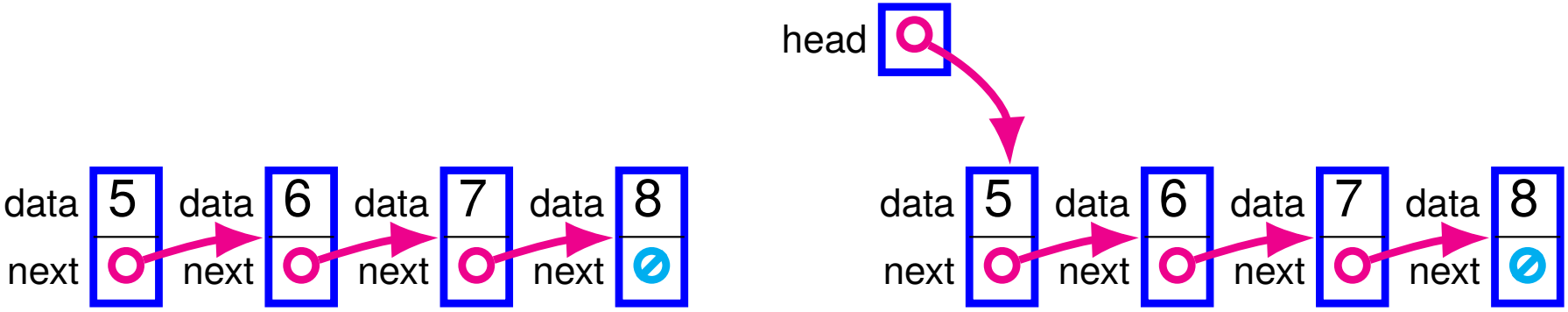
- Change the header to point to the next of the current head

Removing at the beginning



- Change the header to point to the next of the current head
- What happens to the former head?
  - In C, you'd have to explicitly deallocate it
  - In DSSL2 (and Python, and 95% of languages), you don't! (garbage collection)

Headerless vs Headerful lists



Both variants are useful in different circumstances:

- Using a linked list inside some larger data structure?
  - E.g., we will see that with dictionaries and graphs
  - $\rightarrow$ **headerless** is fine
- Using a linked list as a standalone data structure?
  - $\rightarrow$ **headerful** avoids issues with updates

For simplicity, we will generally use **headerless** in examples.

For robustness, we will generally use **headerful** in practice.

Stop! Question time!

Before we move on to an implementation in DSSL2...

Anything unclear so far?

Anything I missed?

Now in DSSL2

Headerless linked lists in DSSL2

```
# Link is one of:
# - cons { data: Any, next: Link }
# - None
struct cons:
    let data
    let next
```

- We use `None` to represent the empty list.
  - Just a convention; we could have used something else
- `cons(1, None)`
  is a list with one element: 1
- `cons(1, cons(2, None))`
  is a list with two elements: 1 and 2
- The number of `cons`es is the number of list elements!

Headerless linked lists in DSSL2

```
# Link is one of:
# - cons { data: Any, next: Link }
# - None
struct cons:
    let data
    let next
```

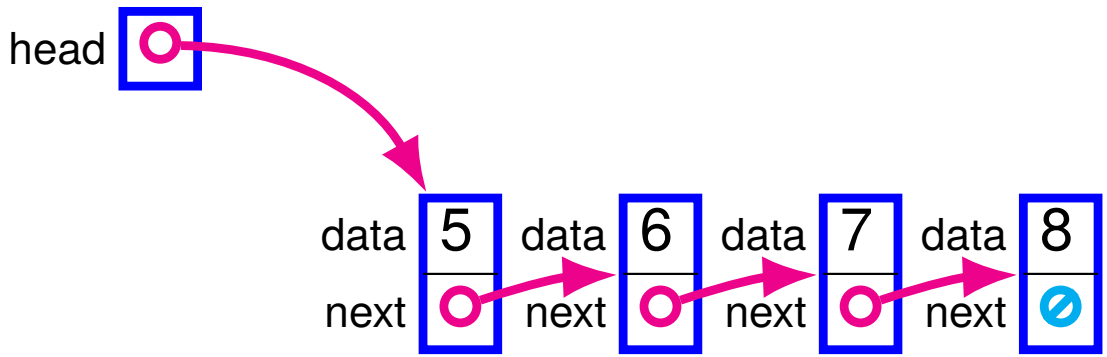
- We use `None` to represent the empty list.
  - Just a convention; we could have used something else
- `cons(1, None)`
  is a list with one element: 1
- `cons(1, cons(2, None))`
  is a list with two elements: 1 and 2
- The number of `cons`es is the number of list elements!

- `cons(None, None)` is a list with one element: `None`
  - I.e., a list of lists with the empty list as its sole element

Headerful linked lists in DSSL2

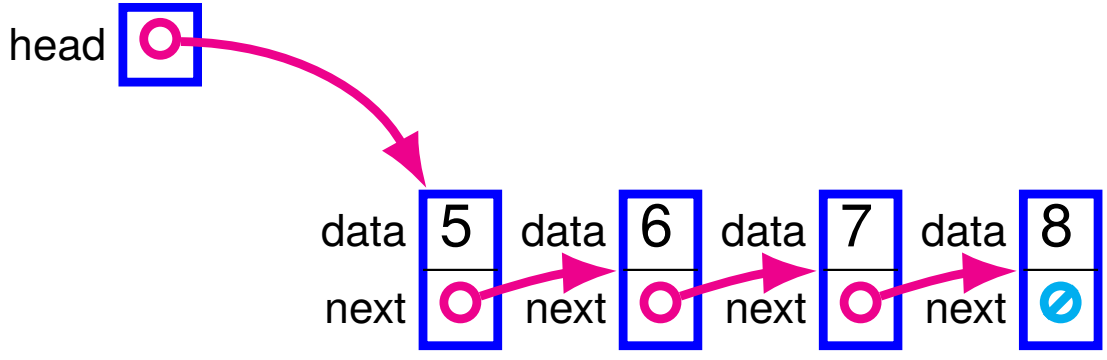
Wrapper around a headerless list

```
class SLL: # SLL = Singly-Linked List
    let head

    def __init__(self):
        self.head = None

    def get_first(self):
        if cons?(self.head): return self.head.data
        else: error('first of an empty list')
```



List operations in DSSL2

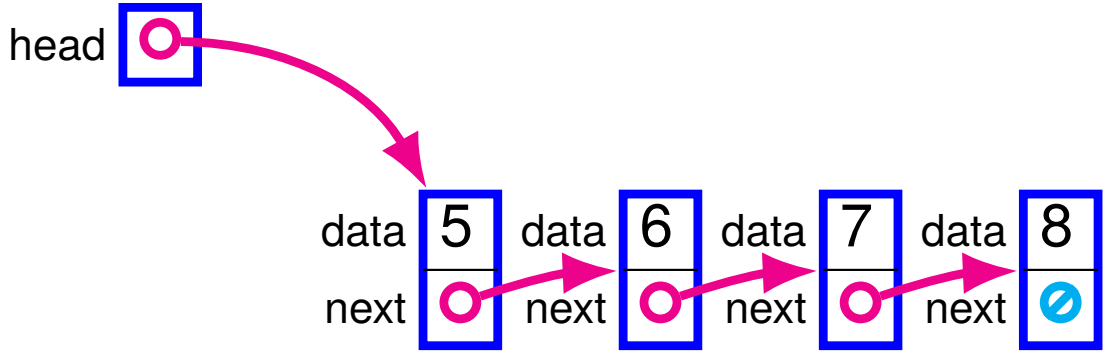
```
class SLL:
    …

    def get_nth(self, n):
        let curr = self.head
        while not curr == None:
            if n == 0:
                return curr.data
            n    = n - 1
            curr = curr.next
        error('list too short')
```



List operations in DSSL2

```
class SLL:
    …

    def get_nth(self, n):
        let curr = self.head
        while not curr == None:
            if n == 0:
                return curr.data
            n    = n - 1
            curr = curr.next
        error('list too short')
```

What about `set_nth`?



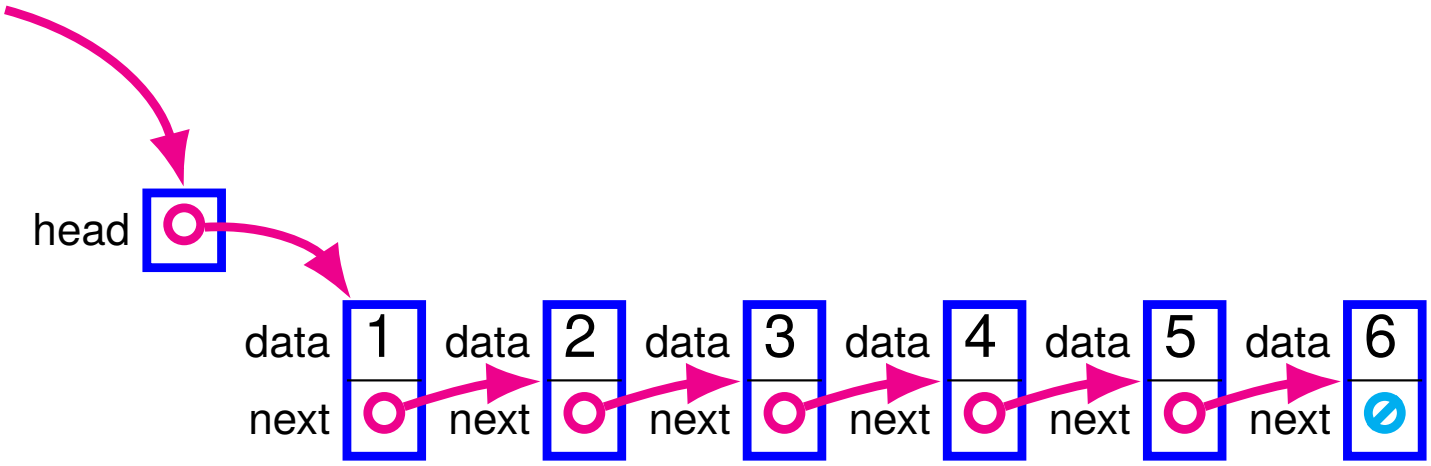
More DSSL2 list operations

```
class SLL:
    …

    def _find_nth_node(self, n):     # leading _ → private
        let curr = self.head
        while not curr == None:
            if n == 0:
                return curr
            n    = n - 1
            curr = curr.next
        error('list too short')

    def get_nth(self, n):
        return self._find_nth_node(n).data
    def set_nth(self, n, val):
        self._find_nth_node(n).data = val
```

What else might we want to do?

- Know how long the list is.
- Insert or remove at the end.
- Etc.

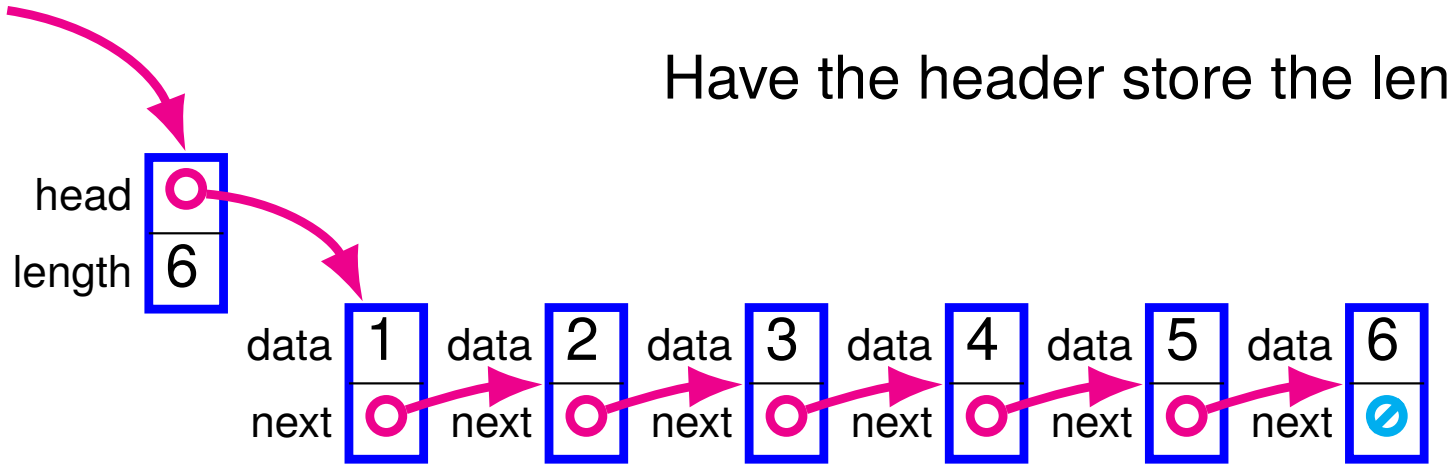
Counting the length




```
def len(self, n):
    let curr = self.head
    let length = 0
    while not curr == None:
        curr = curr.next
        length = length + 1
    return length
```

If we have a very long list, this will take a while!
And we repeat the same work every time we call `len`!

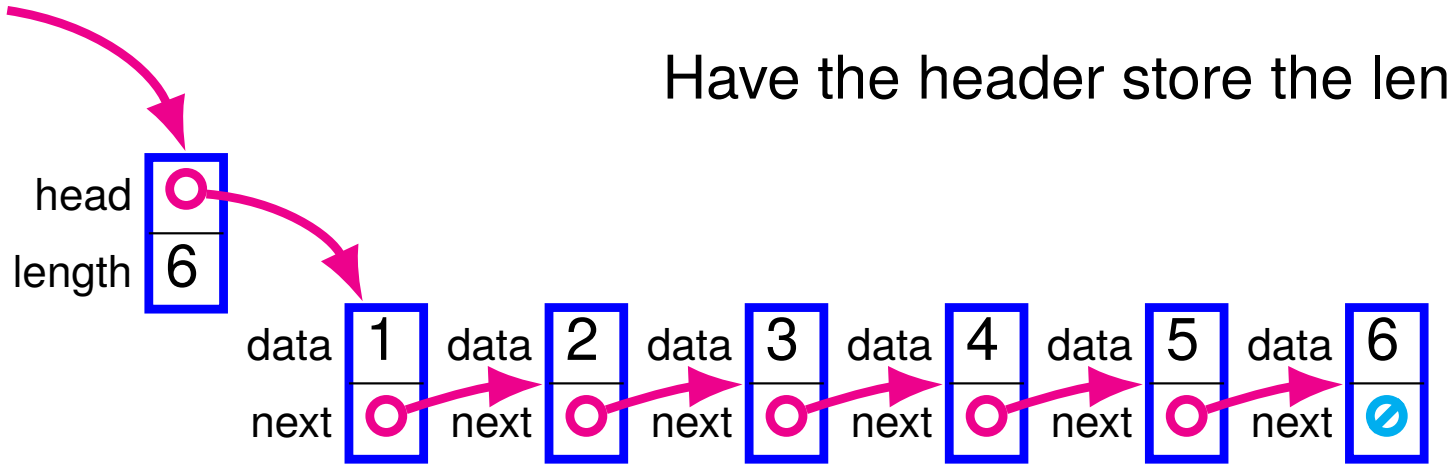
Storing the length



Have the header store the length!

QUIZ: How can we make sure the `length` field is always right?

Storing the length

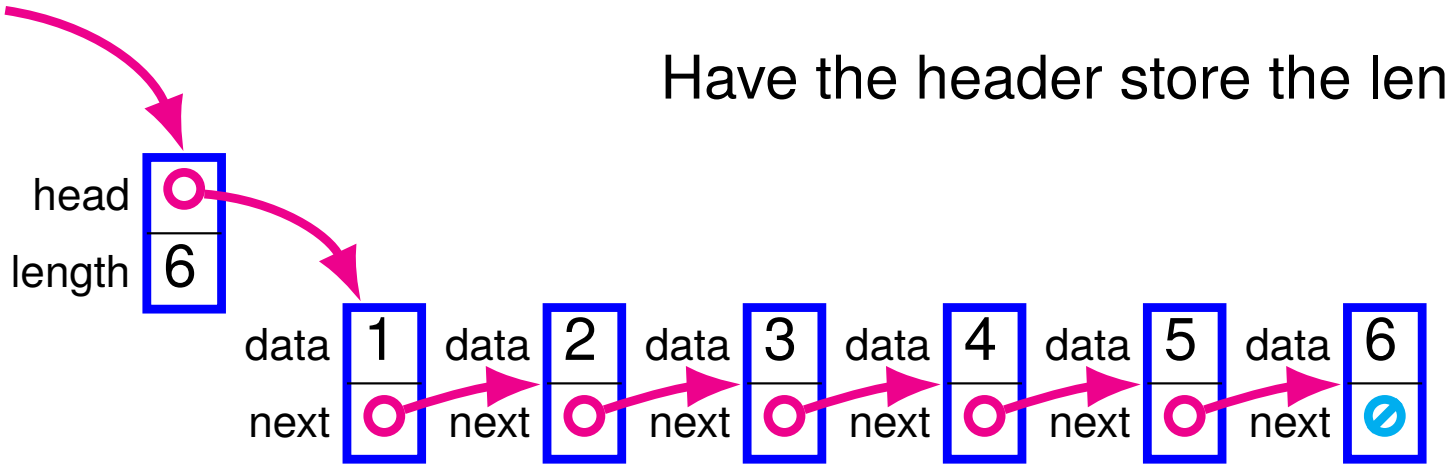


Have the header store the length!

QUIZ: How can we make sure the `length` field is always right?

By having all adds and removes update the length!

Storing the length



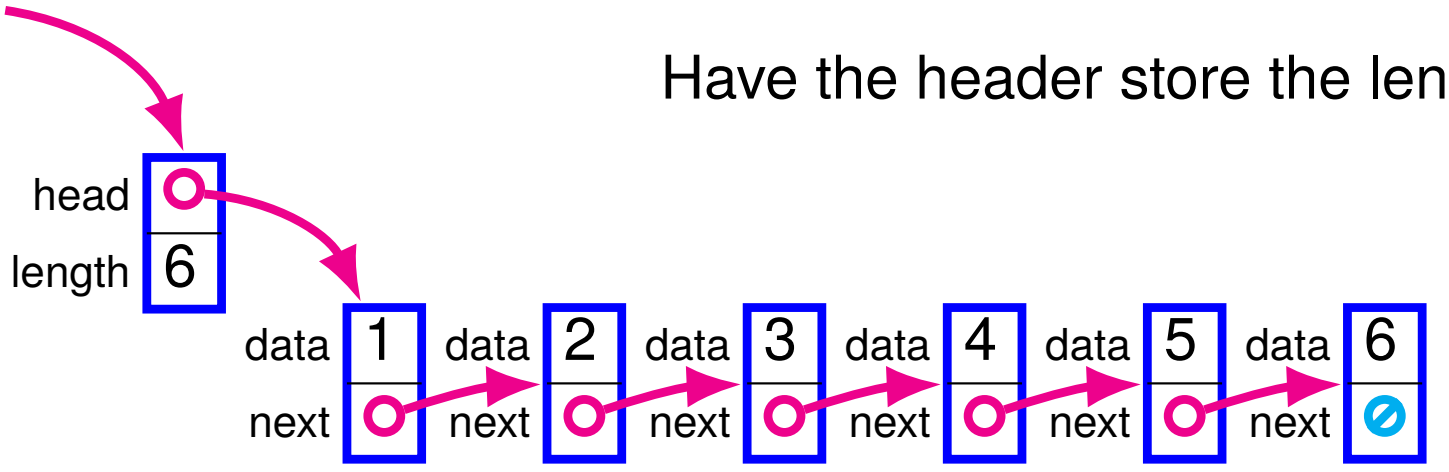
QUIZ: How can we make sure the `length` field is always right?

By having all adds and removes update the length!

Invariant: value of the `length` field = # of nodes in the list.

An *invariant* is a property that a data structure (or algorithm) must satisfy in order to function correctly.

Storing the length



QUIZ: How can we make sure the `length` field is always right?

By having all adds and removes update the length!

Invariant: value of the `length` field = # of nodes in the list.

If an invariant is broken, operations will not work as intended! (Either incorrect or inefficient.)

Previously: binary search needs sorted vectors to be correct.

Your turn: Getting the last element

Download `list-starter.rkt` from Canvas.

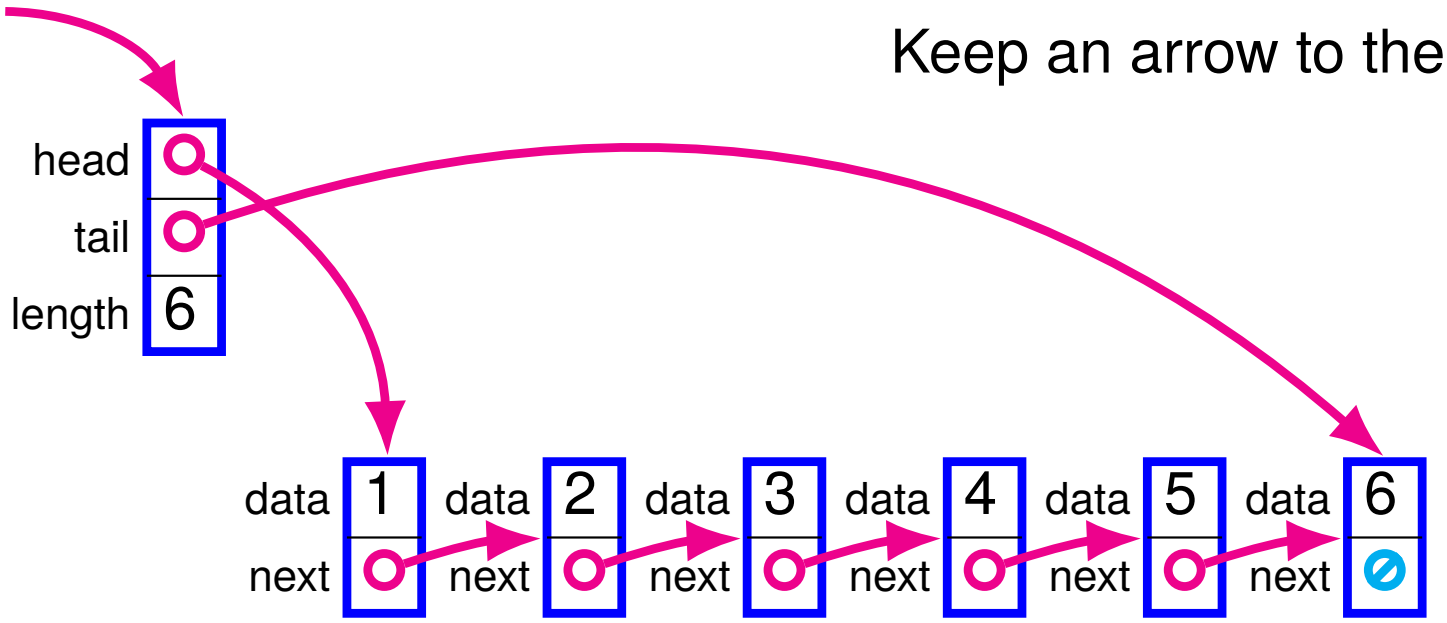
Your task:

- Implement the `get_last` method, which returns the last element of the list.

Work with your neighbor.

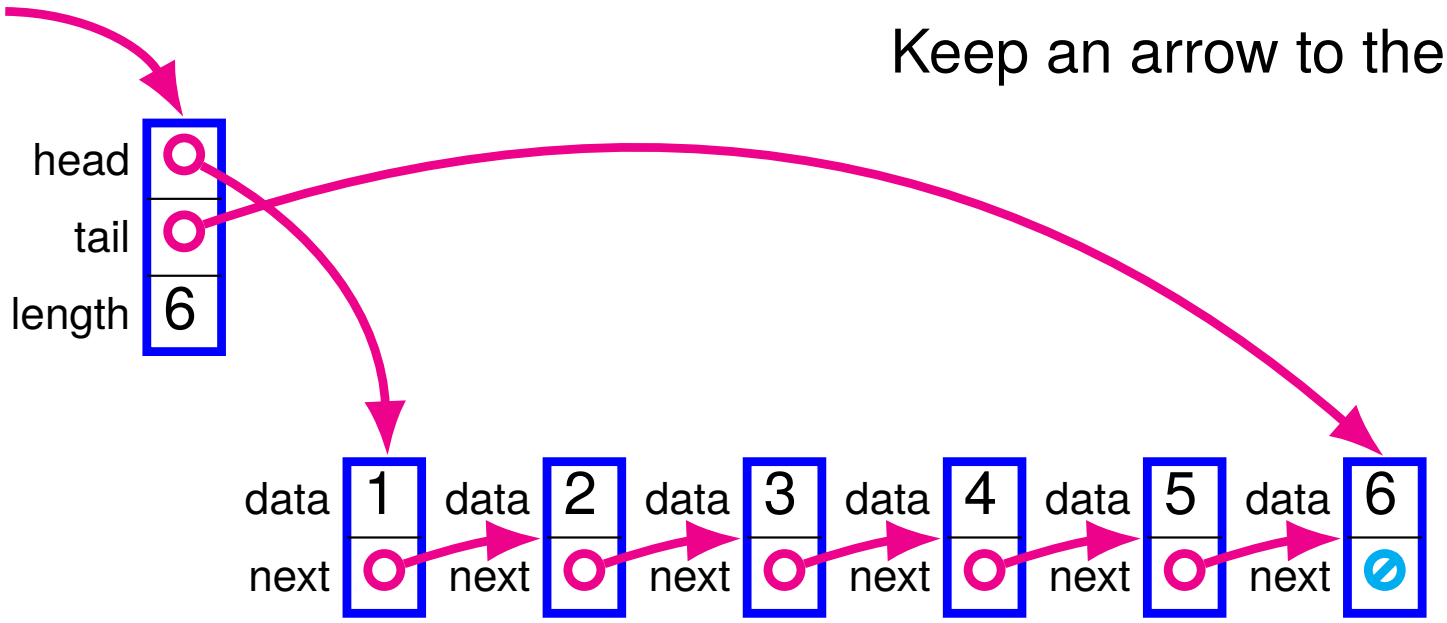
Let's debrief in five minutes.

Quick access to the tail



QUIZ: Name some operations that are less work now.

Quick access to the tail

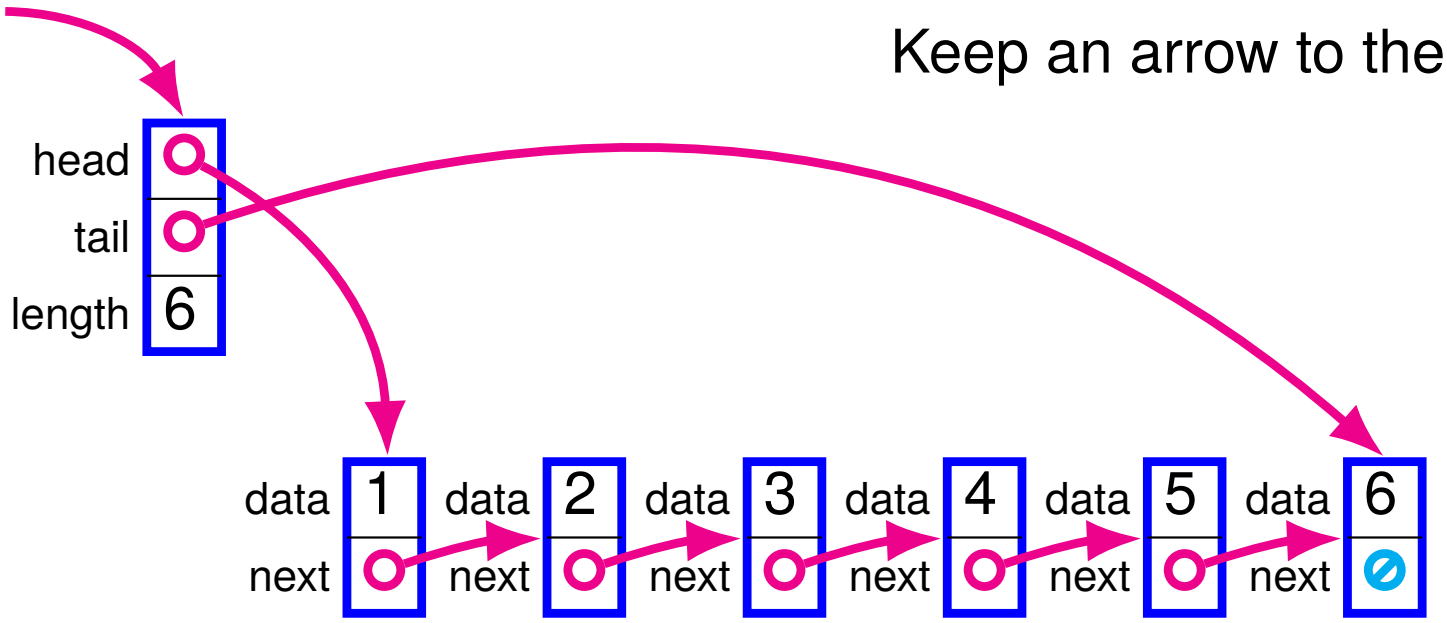


QUIZ: Name some operations that are less work now.

Get/set last, append, etc.

QUIZ: Name some that are still a lot of work.

Quick access to the tail



QUIZ: Name some operations that are less work now.

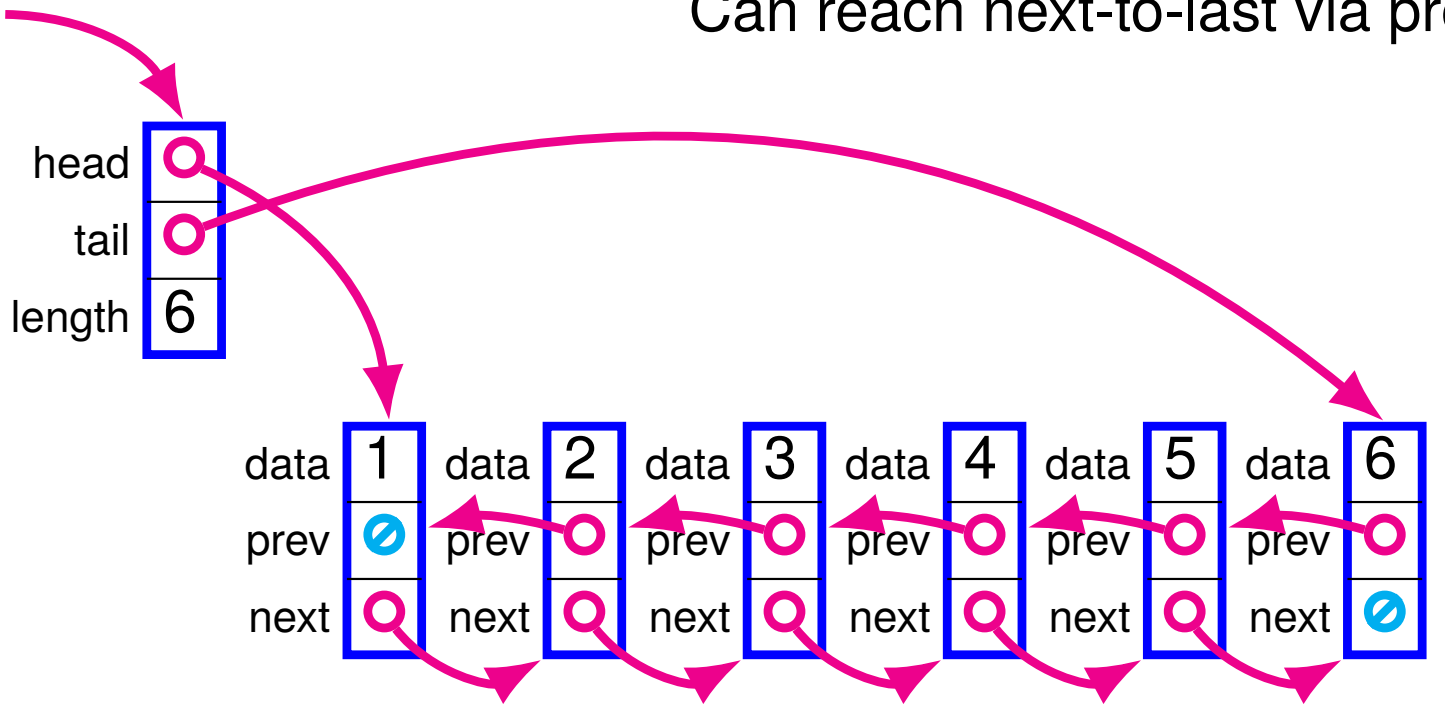
Get/set last, append, etc.

QUIZ: Name some that are still a lot of work.

Get/set *n*th, split, etc. **Also**: remove last!

Doubly-linked lists

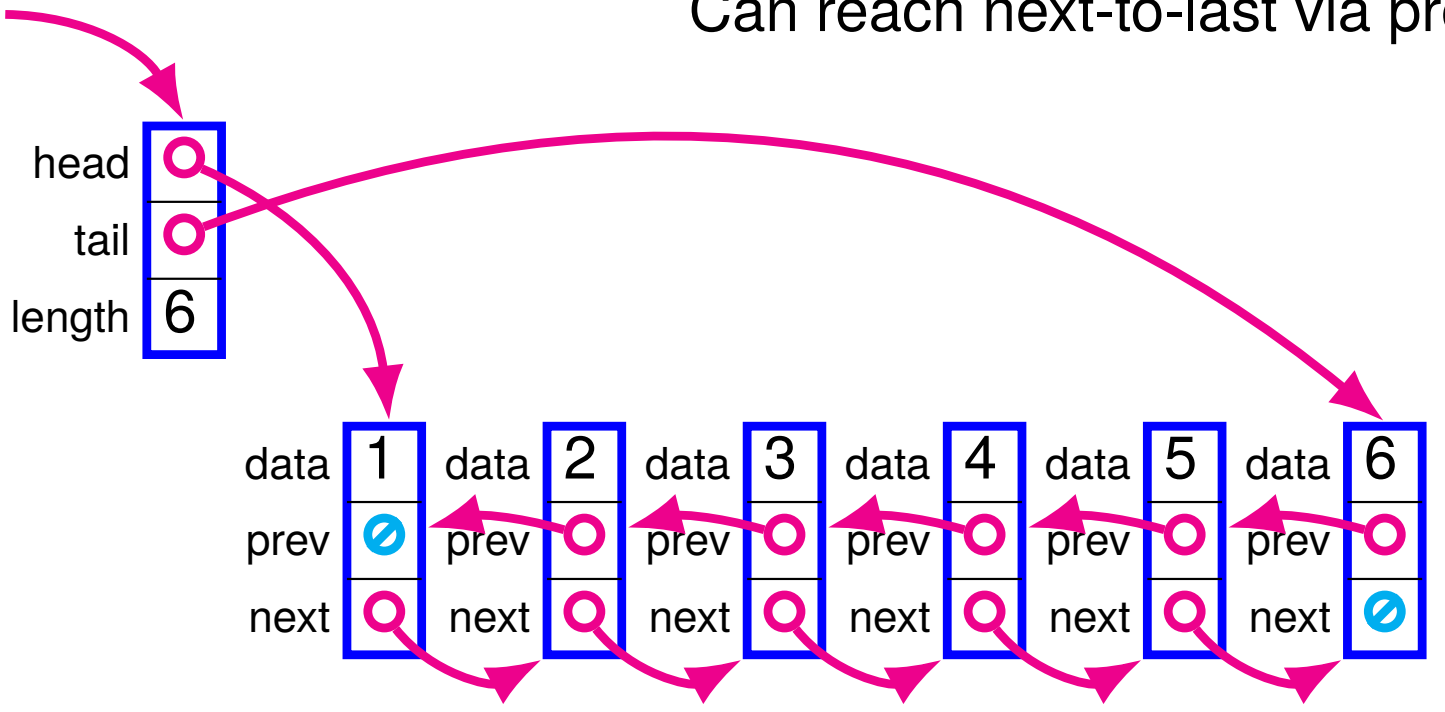
Can reach next-to-last via prev of tail!



QUIZ: What are some invariants for a doubly-linked list?

Doubly-linked lists

Can reach next-to-last via prev of tail!



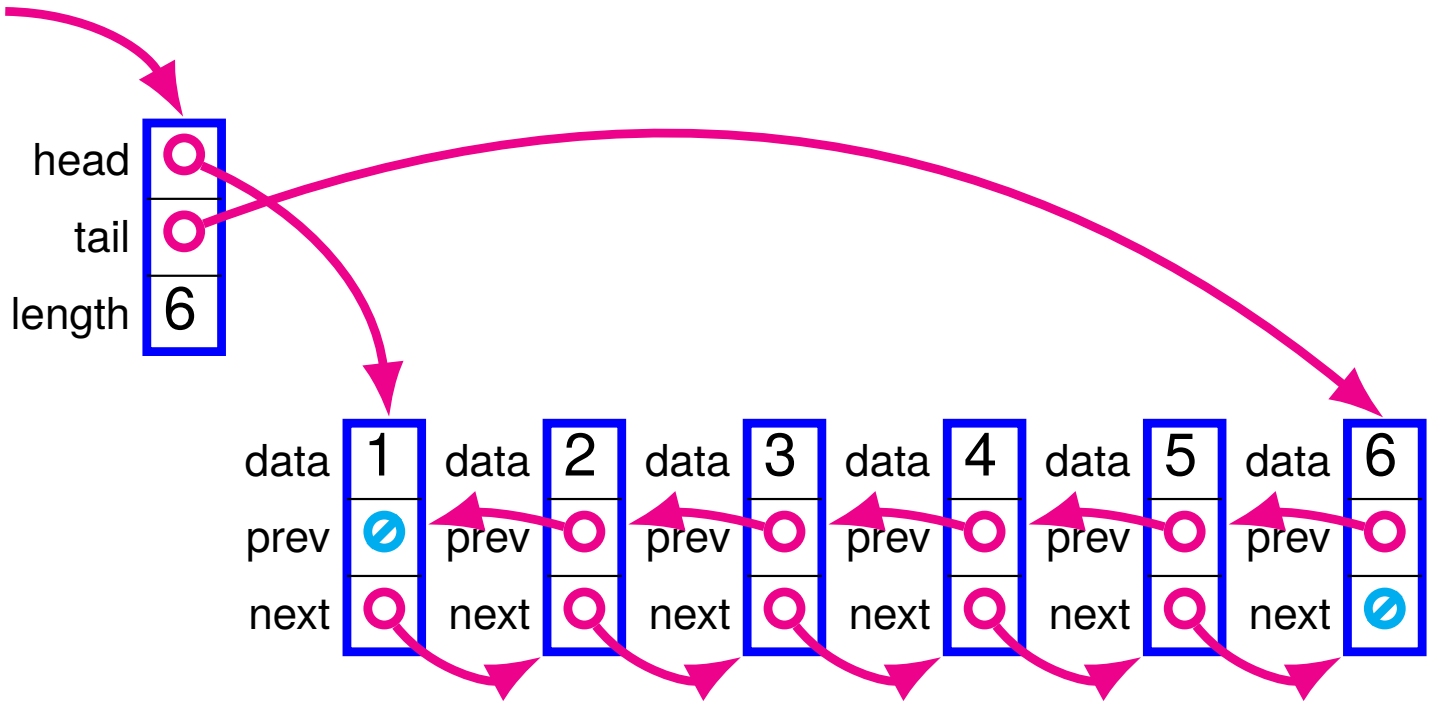
QUIZ: What are some invariants for a doubly-linked list?

`head` has no `prev`. `tail` has no `next`.

The `prev` field of our `next` is ourselves, and vice versa.

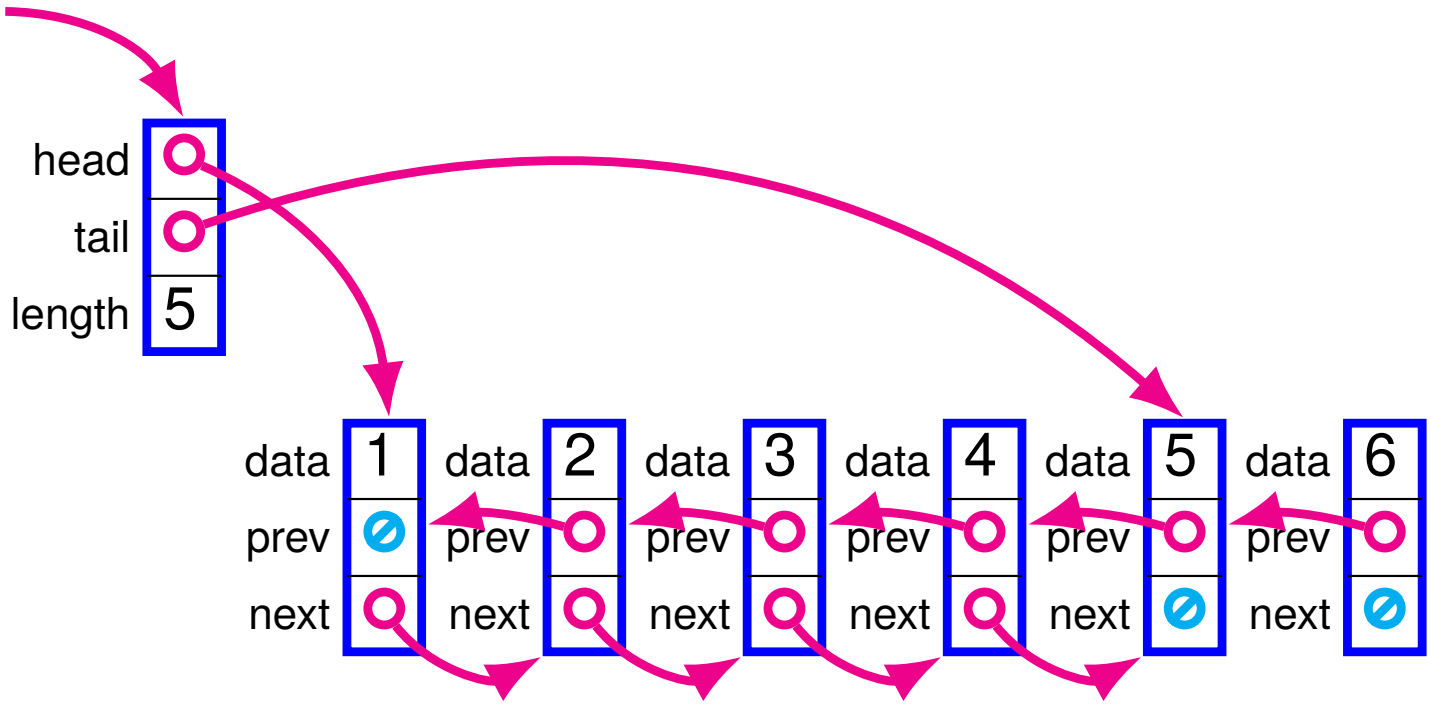
(Lots of edge cases... Tricky to get right.)

Removing at the end



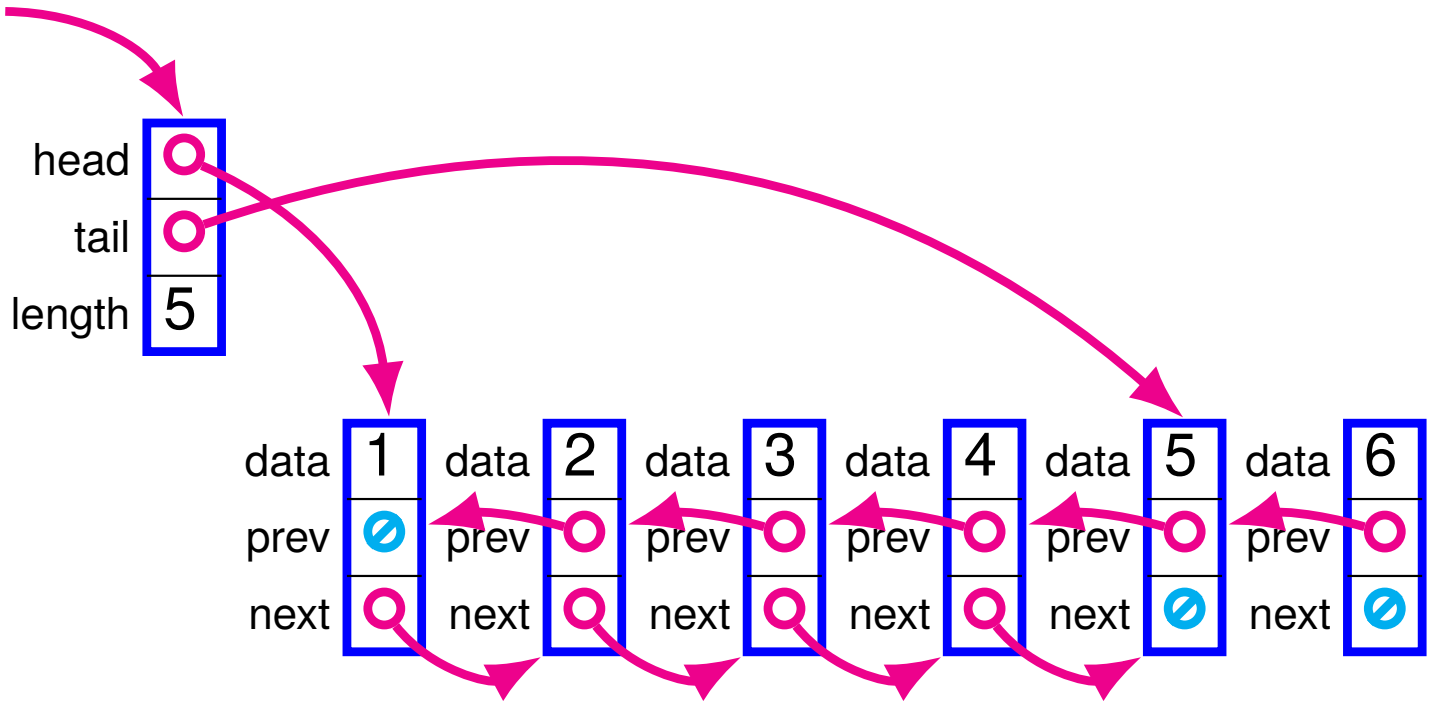
- Pay close attention to this diagram.
- I'll change it, and you'll have to spot the differences.

Removing at the end



QUIZ: what are the three things that changed?

Removing at the end



QUIZ: what are the three things that changed?

- `tail` now points to 5 node.
- `next` of 5 node is now None.
- `length` is decremented.

Invariants galore! Gotta respect 'em all!

Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)

Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)

Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)

Addendum: There is hope for vectors

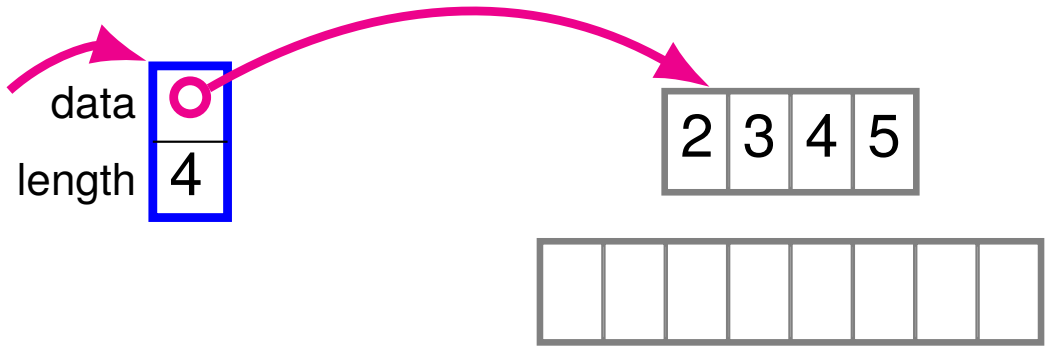
- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)

Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

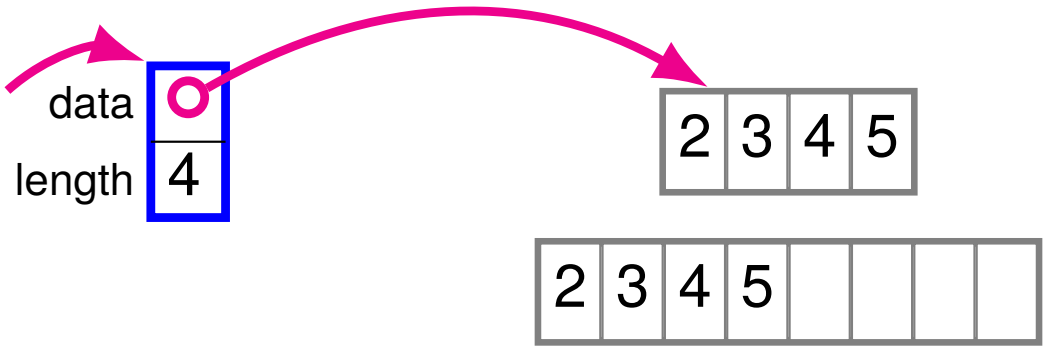
- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

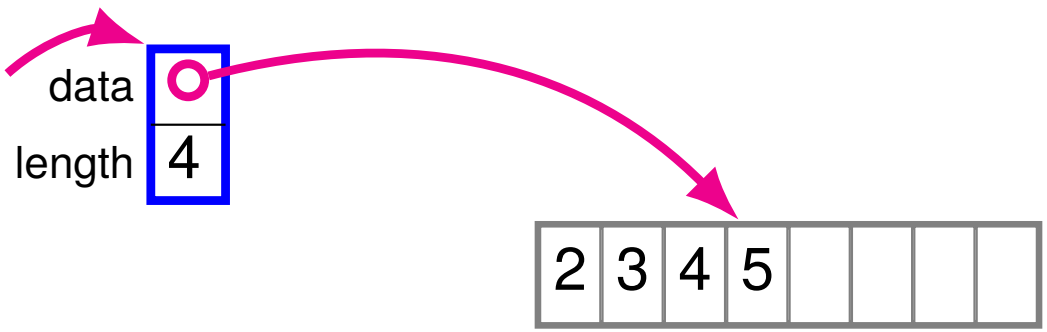
- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

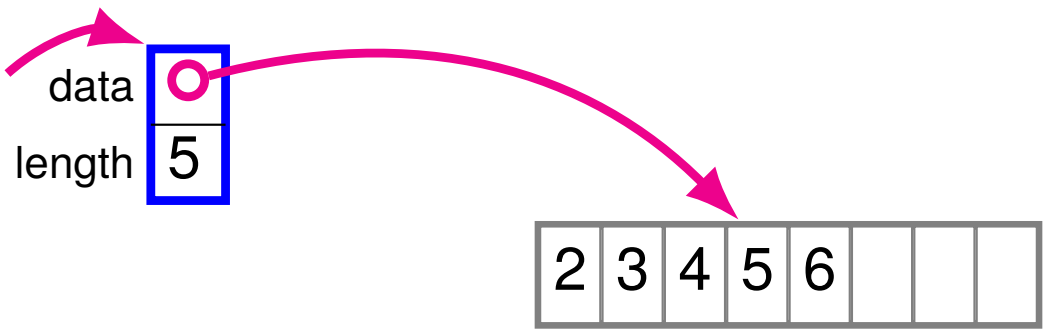
- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

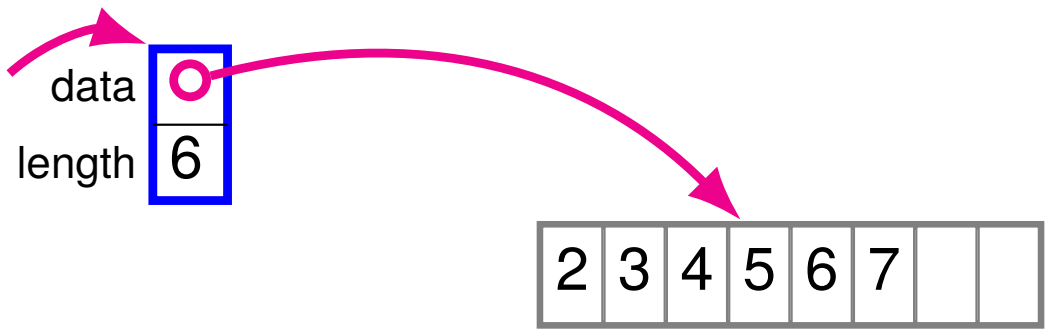
- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

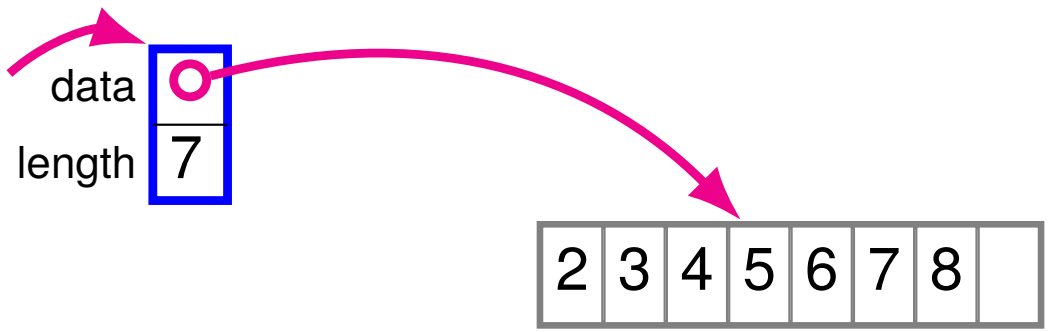
- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

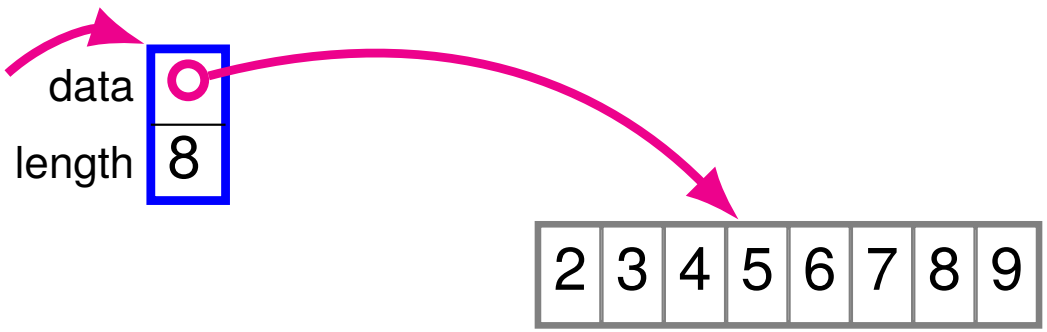
- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

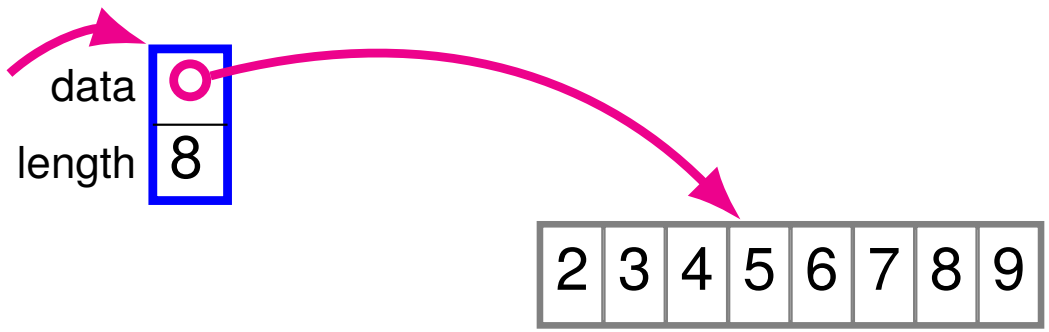
- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



Addendum: There is hope for vectors

- Individual vectors have a fixed size
  - Can't change it after creation
- But we *can* build a growable data structure using vectors!

- **Dynamic arrays**: another data structure!
  - Allocate a vector with some empty space at the end
  - Can add elements to the end until we're full
  - Then allocate bigger vector, copy elements, and keep going
    - Doubling size is efficient (see lectures 7 and 17)



`std::vector` in C++, `ArrayList` in Java, and `list` in Python!

Additional practice: SLL with tail arrow

Download `list-tail-starter.rkt` from Canvas.

Your tasks:

- Rewrite the `get_last` method to use the `tail` arrow.
  - Did you need to change anything else?
- Implement the `len` method using the invariant we saw

Next time: ADTs